

**“End-to-End Big Data  
Analytics on WHO Mortality  
Data Using Apache Spark on  
Cloud”**

# TABLE OF CONTENTS

|                                                   |    |
|---------------------------------------------------|----|
| INTRODUCTION.....                                 | 3  |
| TECHNOLOGIES USED .....                           | 4  |
| • PySpark.....                                    | 4  |
| • AWS S3 .....                                    | 4  |
| • Google Colab .....                              | 4  |
| • WHO Mortality Dataset .....                     | 4  |
| • Power BI .....                                  | 5  |
| SYSTEM ARCHITECTURE (Diagram will be added) ..... | 5  |
| METHODOLOGY & WORKFLOW .....                      | 6  |
| IMPLEMENTATION & CODE .....                       | 7  |
| OUTPUT & SCREENSHOTS.....                         | 9  |
| RESULTS .....                                     | 10 |
| REFERENCES.....                                   | 11 |

## INTRODUCTION

The rapid growth of data across the globe has led to the evolution of new technologies that can efficiently store, process, and analyze large datasets. Health-related data specifically has become extremely valuable for governments, researchers, and public health organizations to understand patterns and make data-driven decisions. With the increase in global health concerns such as epidemics and pandemics, organizations like the World Health Organization (WHO) collect large amounts of mortality data from different countries every year.

Processing such vast data using traditional systems can result in high computation time and limited scalability. This challenge has given rise to Big Data technologies like Apache Spark, which provides distributed computing, in-memory processing, and high fault tolerance. Spark, combined with Python (PySpark), allows fast data transformation and analysis in a simplified programming environment.

On the other hand, storing huge amounts of data securely is equally important. Amazon Web Services (AWS) offers Simple Storage Service (S3), a highly scalable, durable, and cloud-based storage system used widely in modern data engineering pipelines. By integrating Google Colab with Spark and AWS S3, we can upload, process, and store large datasets efficiently on the cloud.

This project focuses on mortality data provided by WHO, specifically related to annual death counts categorized by year, sex, and age groups. The objective is to build a data pipeline where the dataset is cleaned, analyzed using PySpark, and then uploaded to AWS S3 for secure storage.

and future analytical use. This demonstrates a complete real-world data engineering workflow, bridging data analytics and cloud technology.

## TECHNOLOGIES USED

- **PySpark**

PySpark is the Python API for Apache Spark, a powerful distributed computing framework. We use PySpark in this project because the dataset contains several years of global mortality statistics, which can be quite large when processed. Traditional Python or Pandas can become slow or run out of memory, but PySpark executes tasks in parallel across multiple computational cores, significantly improving performance. It enables scalable data cleaning, transformation, filtering, and analysis, making it ideal for real-world data engineering workloads like ours.

- **AWS S3**

Amazon Web Services Simple Storage Service (AWS S3) is used as our cloud storage solution. Instead of keeping data locally, S3 lets us upload, organize, and securely store our input datasets and processed files online. This ensures durability, easy sharing, and accessibility from anywhere. In a professional data engineering pipeline, data typically flows in and out of cloud storage — and S3 is one of the most widely adopted storage solutions for such pipelines. It also integrates smoothly with PySpark using the S3A connector, allowing us to read and write data directly from the cloud.

- **Google Colab**

Google Colab is an online Jupyter notebook environment that supports Python without requiring any complex installation. It provides free compute resources including RAM, GPUs, and pre-installed libraries that simplify development. We use Google Colab as our execution platform because it allows us to run PySpark code, connect to AWS S3, visualize results, and document everything in one place. This makes development faster, portable, and beginner-friendly.

- **WHO Mortality Dataset**

The dataset used in this project is published by the World Health Organization (WHO) and contains records of deaths across multiple countries, categorized by year, age groups, gender, and disease causes. It is a reliable and authoritative source for global health analysis. We chose this dataset because it provides meaningful real-world insights regarding public health issues such as excess deaths during pandemics or disease impact trends. It enables us to demonstrate practical data preprocessing and analytics using PySpark.

- **Power BI**

Power BI is used for the final visualization of the processed dataset. It connects to the Parquet file stored in AWS S3 and converts raw numerical outputs into interactive dashboards. These visualizations help interpret KPIs such as confirmed cases, recovery trends, and country-wise comparisons in a more meaningful and insightful manner.

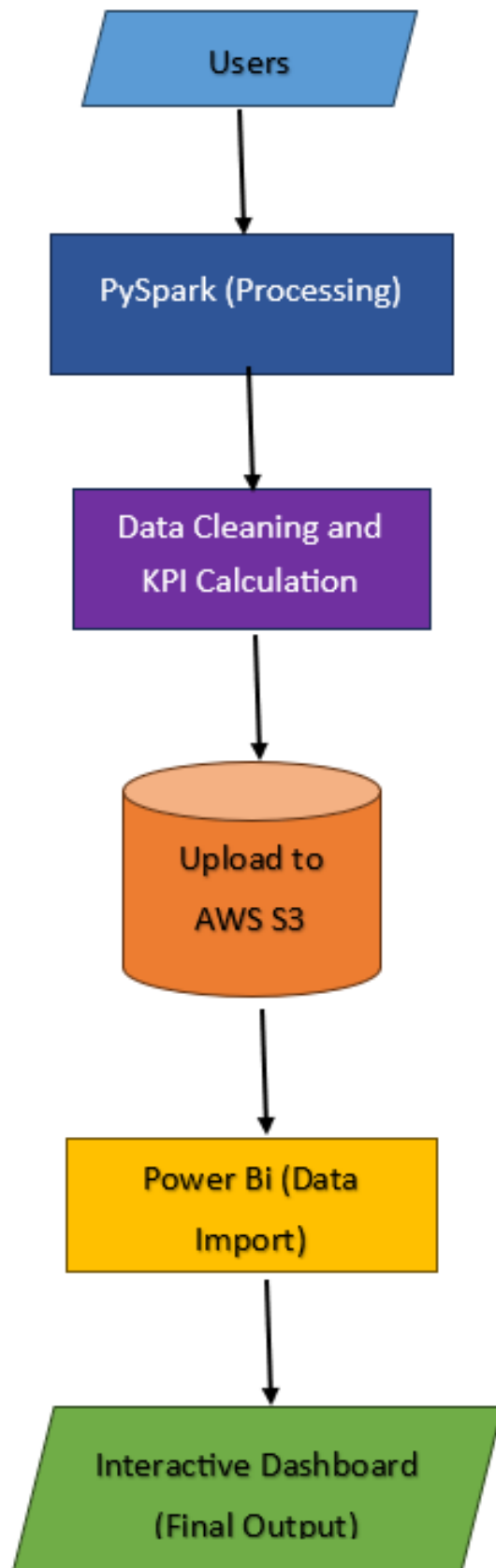
## SYSTEM ARCHITECTURE

*User → PySpark → Data Cleaning & KPI Calculation → Upload to AWS S3 → Power BI → Interactive Dashboards*

The system architecture of this project represents the complete end-to-end flow of data—from input to cloud storage and visualization. It ensures that the dataset is efficiently processed using PySpark, stored securely on AWS S3, and later used for meaningful analytical insights.

The workflow begins with the user uploading the WHO dataset into Google Colab. PySpark handles the data cleaning, transformation, and KPI computation (such as total cases, total deaths, and mortality trends across countries). Once the data is refined and optimized, it is exported to an AWS S3 bucket in Parquet format, enabling high-performance storage and faster analytical queries.

Finally, the processed data stored in S3 is integrated with Power BI to create dynamic and visually interactive dashboards. This end architecture enables scalable data storage, efficient processing, and insightful global health analytics.



# METHODOLOGY & WORKFLOW

The workflow of this project follows a structured data engineering pipeline that processes the WHO dataset and converts it into an optimized analytical format. Each step ensures data quality, scalability, and accessibility for analytics and visualization. The major steps involved are:

- Importing the Dataset

The raw WHO COVID-19 dataset is uploaded into Google Colab. This dataset includes multiple attributes such as country, date, confirmed cases, and deaths. Initially, the data is loaded in CSV format for processing.

- Converting to a Spark DataFrame

Once the CSV is imported, it is converted into a Spark DataFrame using PySpark. A DataFrame structure allows distributed processing across clusters, making the transformation faster and more efficient than using regular Python data structures.

- Data Cleaning & Transformation

- This stage involves:
- Removing missing or invalid records
- Standardizing data fields
- Grouping data by Country
- Aggregating total confirmed cases and death counts
- Calculating KPI metrics needed for analysis

This results in a cleaner and more meaningful dataset that can support future analytics.

- Connecting to AWS S3

Secure authentication is established using IAM Access Key and Secret Key. PySpark's Hadoop S3A library is configured to enable reading and writing data directly to AWS S3. This step ensures that the cloud becomes the final storage destination for processed data.

- Exporting Results to Parquet Format in S3

After transformation, the final DataFrame is exported to AWS S3 in Parquet format.

Why Parquet?

- It provides better compression
- Faster query execution in analytics tools
- Widely supported by BI platforms like Power BI
- This makes the processed data ready for efficient visualization and reporting.

## IMPLEMENTATION & CODE

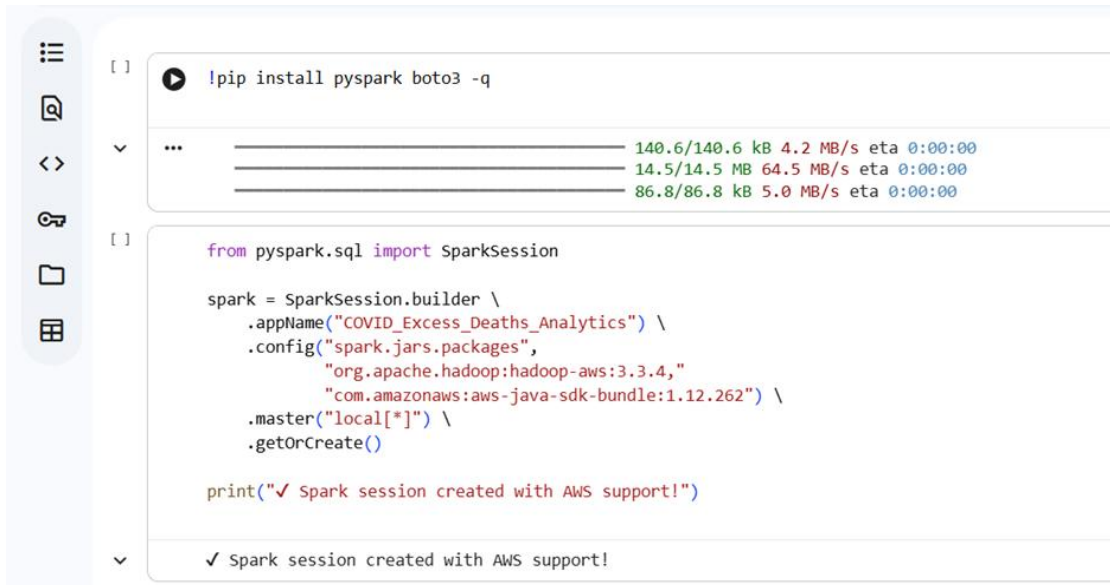
The implementation of this project was carried out using Google Colab, PySpark, and AWS S3. Each step of the data pipeline was coded to ensure seamless processing and cloud storage. The major code-based components are highlighted below:

- **Reading the WHO Dataset**

The raw COVID-19 death count data is uploaded into Google Colab. Using Pandas and PySpark libraries, the dataset is imported and then converted into a Spark DataFrame for distributed processing. This allows handling large datasets efficiently and supports parallel execution.

**Code Highlight:** Import CSV → Display Schema → Basic Data Validation

- Spark Installation



The image shows a Jupyter Notebook interface with a sidebar on the left containing icons for file operations. The main area displays two code cells. The first cell contains a terminal command to install pip packages. The second cell contains Python code to create a SparkSession with AWS support. The output of the second cell shows a confirmation message.

```
[ ] !pip install pyspark boto3 -q
```

```
... 140.6/140.6 kB 4.2 MB/s eta 0:00:00  
14.5/14.5 MB 64.5 MB/s eta 0:00:00  
86.8/86.8 kB 5.0 MB/s eta 0:00:00
```

```
[ ] from pyspark.sql import SparkSession  
  
spark = SparkSession.builder \  
    .appName("COVID_Excess_Deaths_Analytics") \  
    .config("spark.jars.packages",  
            "org.apache.hadoop:hadoop-aws:3.3.4,"  
            "com.amazonaws:aws-java-sdk-bundle:1.12.262") \  
    .master("local[*]") \  
    .getOrCreate()  
  
print("✓ Spark session created with AWS support!")
```

```
✓ Spark session created with AWS support!
```

- Spark configured for S3



The image shows a Jupyter Notebook interface with a sidebar on the left containing icons for file operations. The main area displays a single code cell containing Python code to configure Spark for S3. The code sets AWS credentials and S3 configuration properties, then prints a confirmation message. The output shows the confirmation message.

```
[ ] AWS_ACCESS_KEY = "  
AWS_SECRET_KEY = "  
AWS_REGION = "ap-south-1"  
BUCKET_NAME = "covid-excess-deaths-akne"  
  
hadoopConf = spark._jsc.hadoopConfiguration()  
hadoopConf.set("fs.s3a.access.key", AWS_ACCESS_KEY)  
hadoopConf.set("fs.s3a.secret.key", AWS_SECRET_KEY)  
hadoopConf.set("fs.s3a.endpoint", f"s3.{AWS_REGION}.amazonaws.com")  
hadoopConf.set("fs.s3a.impl", "org.apache.hadoop.fs.s3a.S3AFileSystem")  
  
print("✓ Spark is now configured for S3!")
```

```
✓ Spark is now configured for S3!
```

- Load raw csv from s3

```
obj = s3.get_object(Bucket="covid-excess-deaths-akne", Key="raw/who_deaths_by_year_sex_age.csv")
pdf = pd.read_csv(obj['Body'], skiprows=7) # FIXED
print(pdf.head())
```

```

      acm.mean Estimated deaths from all-causes by age, sex and year (mean) \
0  excess.mean* Excess deaths associated with COVID-19 pandemi...
1      NaN                                           NaN
2      country                                           iso3
3  Afghanistan                                           AFG
4  Afghanistan                                           AFG

      Unnamed: 2 Unnamed: 3 Unnamed: 4 Unnamed: 5      Unnamed: 6 \
0      NaN      NaN      NaN      NaN      NaN
1      NaN      NaN      NaN      NaN      NaN
2      year      sex  age_group      type  expected.mean
3      2020      Female      0-24  predicted  49083.6439335266
4      2020      Female      25-34  predicted  6452.9670394984

      Unnamed: 7      Unnamed: 8
0      NaN      NaN
1      NaN      NaN
2      acm.mean      excess.mean*
3  49103.1431533225      0
4  6691.24721854869  236.607817172177
```

## ● Creating & Transforming Spark DataFrame

PySpark is used to perform data cleaning and transformation including:

- Handling missing values
- Standardizing formats
- Aggregations country-wise
- Preparing KPI-ready structured data

This ensures that the dataset is optimized for storage and analytics.

**Code Highlight:** GroupBy Country → Aggregate Cases & Death Counts → Rename Columns

- Convert to spark Dataframe

```
sex_age_df = spark.createDataFrame(pdf)
sex_age_df.printSchema()
sex_age_df.show(5)
```

```
root
|-- country: string (nullable = true)
|-- iso3: string (nullable = true)
|-- year: string (nullable = true)
|-- sex: string (nullable = true)
|-- age_group: string (nullable = true)
|-- type: string (nullable = true)
|-- expected_mean: string (nullable = true)
|-- acm_mean: string (nullable = true)
|-- excess_mean: string (nullable = true)
```

| country      | iso3                 | year | sex    | age_group | type      | expected_mean    | acm_mean         | excess_mean      |
|--------------|----------------------|------|--------|-----------|-----------|------------------|------------------|------------------|
| excess.mean* | Excess deaths ass... | NaN  | NaN    | NaN       | NaN       | NaN              | NaN              | NaN              |
| NaN          | NaN                  | NaN  | NaN    | NaN       | NaN       | NaN              | NaN              | NaN              |
| country      | iso3                 | year | sex    | age_group | type      | expected_mean    | acm_mean         | excess_mean*     |
| Afghanistan  | AFG                  | 2020 | Female | 0-24      | predicted | 49083.6439335266 | 49103.1431533225 | 0                |
| Afghanistan  | AFG                  | 2020 | Female | 25-34     | predicted | 6452.9670394984  | 6691.24721854869 | 236.607817172177 |

only showing top 5 rows

- Data Cleaning (removing useless header rows)

```
# Remove rows where year = 'year' or country contains description text
clean_df = sex_age_df.filter(sex_age_df.year != "year")

# Also remove any completely blank rows
clean_df = clean_df.dropna(subset=["country", "year", "sex"])

clean_df.show(10)
clean_df.printSchema()
```

```
root
|-- country: string (nullable = true)
|-- iso3: string (nullable = true)
|-- year: string (nullable = true)
```

| country      | iso3                 | year | sex    | age_group | type      | expected_mean    | acm_mean         | excess_mean      |
|--------------|----------------------|------|--------|-----------|-----------|------------------|------------------|------------------|
| excess.mean* | Excess deaths ass... | NaN  | NaN    | NaN       | NaN       | NaN              | NaN              | NaN              |
| NaN          | NaN                  | NaN  | NaN    | NaN       | NaN       | NaN              | NaN              | NaN              |
| Afghanistan  | AFG                  | 2020 | Female | 0-24      | predicted | 49083.6439335266 | 49103.1431533225 | 0                |
| Afghanistan  | AFG                  | 2020 | Female | 25-34     | predicted | 6452.9670394984  | 6691.24721854869 | 236.607817172177 |
| Afghanistan  | AFG                  | 2020 | Female | 35-44     | predicted | 6117.87310635972 | 6977.363938828   | 860.30071392836  |
| Afghanistan  | AFG                  | 2020 | Female | 45-54     | predicted | 7711.68953142234 | 9330.21731710218 | 1621.57180617288 |
| Afghanistan  | AFG                  | 2020 | Female | 55-64     | predicted | 10061.5441565976 | 12457.9850857855 | 2401.48897102767 |
| Afghanistan  | AFG                  | 2020 | Female | 65-74     | predicted | 13955.3421144752 | 17144.299885298  | 3195.44937263931 |
| Afghanistan  | AFG                  | 2020 | Female | 75-84     | predicted | 12751.7113222847 | 14639.0412442922 | 1889.38041140226 |
| Afghanistan  | AFG                  | 2020 | Female | >85       | predicted | 3694.55430771967 | 4614.09830510881 | 921.54726233789  |

only showing top 10 rows

```
root
|-- country: string (nullable = true)
|-- iso3: string (nullable = true)
|-- year: string (nullable = true)
```

- Keep only numeric year

```

▶ from pyspark.sql.functions import col

# Keep rows where year is numeric only
clean_df = clean_df.filter(col("year").rlike("^[0-9]{4}$"))

clean_df.show(10)
clean_df.printSchema()

```

```

*** +-----+-----+-----+-----+-----+-----+-----+-----+
| country|iso3|year| sex|age_group| type| expected_mean| acm_mean| excess_mean|
+-----+-----+-----+-----+-----+-----+-----+-----+
|Afghanistan| AFG|2020|Female| 0-24|predicted|49083.6439335266|49103.1431533225| 0|
|Afghanistan| AFG|2020|Female| 25-34|predicted| 6452.9670394984|6691.24721854869|236.607817172177|
|Afghanistan| AFG|2020|Female| 35-44|predicted|6117.87310635972| 6977.363938828| 860.30071392836|
|Afghanistan| AFG|2020|Female| 45-54|predicted|7711.68953142234|9330.21731710218|1621.57180617288|
|Afghanistan| AFG|2020|Female| 55-64|predicted|10061.5441565976|12457.9850857855|2401.48897102767|
|Afghanistan| AFG|2020|Female| 65-74|predicted|13955.3421144752| 17144.299885298|3195.44937263931|
|Afghanistan| AFG|2020|Female| 75-84|predicted|12751.7113222847|14639.0412442922|1889.38041140226|
|Afghanistan| AFG|2020|Female| >85|predicted|3694.55430771967|4614.09830510881|921.547262333789|
|Afghanistan| AFG|2020| Male| 0-24|predicted| 67685.668415993|67712.5575713145| 0|
|Afghanistan| AFG|2020| Male| 25-34|predicted|15363.9268330534|15618.5418317011|249.450996723866|
+-----+-----+-----+-----+-----+-----+-----+-----+
only showing top 10 rows

root
|-- country: string (nullable = true)
|-- iso3: string (nullable = true)
|-- year: string (nullable = true)
|-- sex: string (nullable = true)
|-- age_group: string (nullable = true)

```

## - Convert Text Columns to Numbers

```

▶ from pyspark.sql.types import DoubleType
from pyspark.sql.functions import col

numeric_df = clean_df.withColumn("expected_mean", col("expected_mean").cast(DoubleType())) \
    .withColumn("acm_mean", col("acm_mean").cast(DoubleType())) \
    .withColumn("excess_mean", col("excess_mean").cast(DoubleType()))

numeric_df.printSchema()
numeric_df.show(5)

```

```

*** root
|-- country: string (nullable = true)
|-- iso3: string (nullable = true)
|-- year: string (nullable = true)
|-- sex: string (nullable = true)
|-- age_group: string (nullable = true)
|-- type: string (nullable = true)
|-- expected_mean: double (nullable = true)
|-- acm_mean: double (nullable = true)
|-- excess_mean: double (nullable = true)

```

| country     | iso3 | year | sex    | age_group | type      | expected_mean    | acm_mean         | excess_mean      |
|-------------|------|------|--------|-----------|-----------|------------------|------------------|------------------|
| Afghanistan | AFG  | 2020 | Female | 0-24      | predicted | 49083.6439335266 | 49103.1431533225 | 0.0              |
| Afghanistan | AFG  | 2020 | Female | 25-34     | predicted | 6452.9670394984  | 6691.24721854869 | 236.607817172177 |
| Afghanistan | AFG  | 2020 | Female | 35-44     | predicted | 6117.87310635972 | 6977.363938828   | 860.30071392836  |
| Afghanistan | AFG  | 2020 | Female | 45-54     | predicted | 7711.68953142234 | 9330.21731710218 | 1621.57180617288 |
| Afghanistan | AFG  | 2020 | Female | 55-64     | predicted | 10061.5441565976 | 12457.9850857855 | 2401.48897102767 |

## - AWS S3 Integration & Export

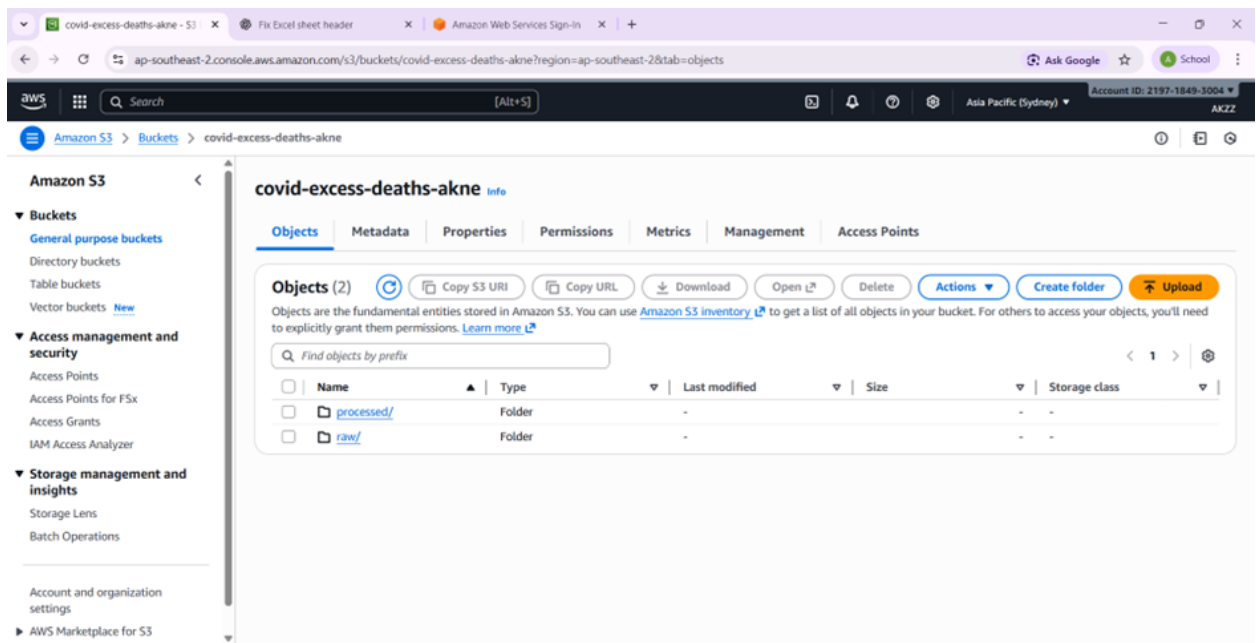
A secure connection to AWS S3 is configured using IAM credentials.

The transformed data is exported to the cloud in Parquet format, offering:

- Reduced storage size
- Improved query performance
- Compatibility with BI tools like Power BI

### Code Highlight:

- ★ Configure Spark to access S3
- ★ Write DataFrame to S3 bucket in Parquet format
- **S3 Bucket**



- Inside Raw File

← → ↻

ap-southeast-2.console.aws.amazon.com/s3/buckets/covid-excess-deaths-akne?region=ap-southeast-2&prefix=raw/&showversions=false

Ask Google ☆ School

AWS

Search [Alt+S]

Asia Pacific (Sydney)

Account ID: 2197-1849-3004 AKZ2

Amazon S3

Buckets

covid-excess-deaths-akne

raw/

Amazon S3

<

▼ Buckets

General purpose buckets

Directory buckets

Table buckets

Vector buckets [New](#)

▼ Access management and security

Access Points

Access Points for FSx

Access Grants

IAM Access Analyzer

▼ Storage management and insights

Storage Lens

Batch Operations

Account and organization settings

API Explorer for FS

raw/

Copy S3 URI

Objects

Properties

Objects (4)

Copy S3 URI

Copy URL

Download

Open i

Delete

Actions

Create folder

Upload

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

Find objects by prefix

< 1 >

| <input type="checkbox"/> | Name                                                            | Type | Last modified                          | Size     | Storage class |
|--------------------------|-----------------------------------------------------------------|------|----------------------------------------|----------|---------------|
| <input type="checkbox"/> | <a href="#">WHO_COVID_Excess_Deaths_EstimatesByCountry.xlsx</a> | xlsx | December 7, 2025, 16:19:35 (UTC+05:30) | 1.0 MB   | Standard      |
| <input type="checkbox"/> | <a href="#">who_deaths_by_year_month.csv</a>                    | csv  | December 7, 2025, 17:29:40 (UTC+05:30) | 451.2 KB | Standard      |
| <input type="checkbox"/> | <a href="#">who_deaths_by_year_sex_age.csv</a>                  | csv  | December 7, 2025, 17:29:41 (UTC+05:30) | 550.9 KB | Standard      |
| <input type="checkbox"/> | <a href="#">who_deaths_by_year.csv</a>                          | csv  | December 7, 2025, 17:29:39 (UTC+05:30) | 33.6 KB  | Standard      |

## OUTPUT & SCREENSHOTS

- Create KPI by Country

```
from pyspark.sql.functions import sum as _sum, avg as _avg, round as _round, col

country_kpi_df = numeric_df.groupBy("country").agg(
    _sum("excess_mean").alias("total_excess_deaths"),
    _avg("excess_mean").alias("avg_excess_deaths")
)

country_kpi_df = country_kpi_df.withColumn("total_excess_deaths", _round(col("total_excess_deaths"), 2))
country_kpi_df = country_kpi_df.withColumn("avg_excess_deaths", _round(col("avg_excess_deaths"), 2))

country_kpi_df.show(10)
```

```
... +-----+-----+-----+
|      country|total_excess_deaths|avg_excess_deaths|
+-----+-----+-----+
|Micronesia (Feder...|          75.0|           2.34|
|      Cabo Verde|        634.19|          19.82|
|      Kiribati|         161.3|           5.04|
| Republic of Korea|       9628.71|         300.9|
|      Guyana|       3821.23|        119.41|
|      Eritrea|       2040.74|         63.77|
|      Djibouti|       1158.41|          36.2|
|      Fiji|       1722.87|         53.84|
|      Iraq|      44847.05|       1401.47|
|      Germany|     101504.97|      3172.03|
+-----+-----+-----+
only showing top 10 rows
```

- Create KPI by Country + Sex

```
country_sex_kpi_df = numeric_df.groupBy("country", "sex").agg(
    _sum("excess_mean").alias("total_excess_deaths")
)

country_sex_kpi_df = country_sex_kpi_df.withColumn(
    "total_excess_deaths", _round(col("total_excess_deaths"), 2)
)

country_sex_kpi_df.show(10)
```

```
... +-----+-----+-----+
|          country|    sex|total_excess_deaths|
+-----+-----+-----+
|          Gabon|Female|          457.33|
|        Honduras|  Male|        11632.36|
|          Japan|  Male|       58340.85|
|          Jordan|Female|        6583.18|
|Antigua and Barbuda|Female|         -34.57|
|    CÃ´te d'Ivoire|Female|        6847.86|
|Iran (Islamic Rep...|  Male|       157227.06|
|        Honduras|Female|         8273.77|
|Bolivia (Plurinatio...|  Male|        56728.34|
|          Germany|Female|       46698.62|
+-----+-----+-----+
only showing top 10 rows
```

- Create KPI by Country + AgeGroup

```
country_age_kpi_df = numeric_df.groupBy("country", "age_group").agg(
    _sum("excess_mean").alias("total_excess_deaths")
)

country_age_kpi_df = country_age_kpi_df.withColumn(
    "total_excess_deaths", _round(col("total_excess_deaths"), 2)
)

country_age_kpi_df.show(10)
```

```
... +-----+-----+-----+
|          country|age_group|total_excess_deaths|
+-----+-----+-----+
|Democratic Republ...| 45-54|      10827.78|
|      Afghanistan| 35-44|       7215.64|
|      Georgia| 35-44|        155.61|
|      Jordan| 75-84|       3490.25|
|      Kazakhstan| 75-84|      22594.22|
|      Eritrea| 0-24|           0.0|
|      Indonesia| 65-74|     301550.87|
|      Andorra| 25-34|         -0.04|
|      Bahamas| 55-64|        286.73|
|Democratic Republ...| 55-64|     13686.35|
+-----+-----+-----+
only showing top 10 rows
```

- Processed KPI files uploaded to S3

```

import pandas as pd

# Convert to Pandas
country_kpi_pdf = country_kpi_df.toPandas()
country_sex_pdf = country_sex_kpi_df.toPandas()
country_age_pdf = country_age_kpi_df.toPandas()

# Save locally
country_kpi_pdf.to_csv("country_kpi.csv", index=False)
country_sex_pdf.to_csv("country_sex_kpi.csv", index=False)
country_age_pdf.to_csv("country_age_kpi.csv", index=False)

# Upload to S3
s3.upload_file("country_kpi.csv", BUCKET_NAME, "processed/country_kpi.csv")
s3.upload_file("country_sex_kpi.csv", BUCKET_NAME, "processed/country_sex_kpi.csv")
s3.upload_file("country_age_kpi.csv", BUCKET_NAME, "processed/country_age_kpi.csv")

print("Processed KPI files uploaded to S3!")

```

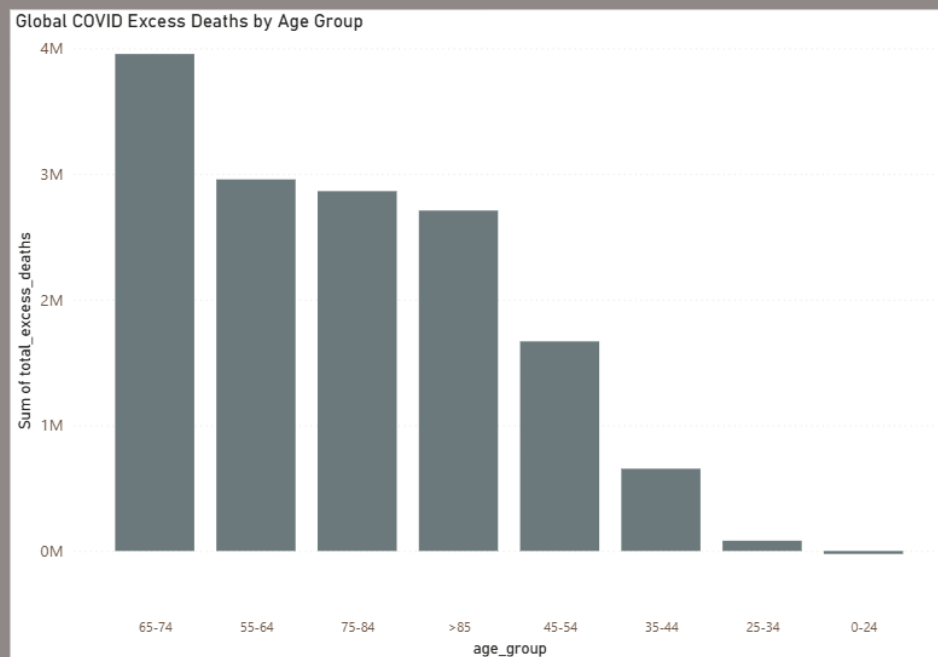
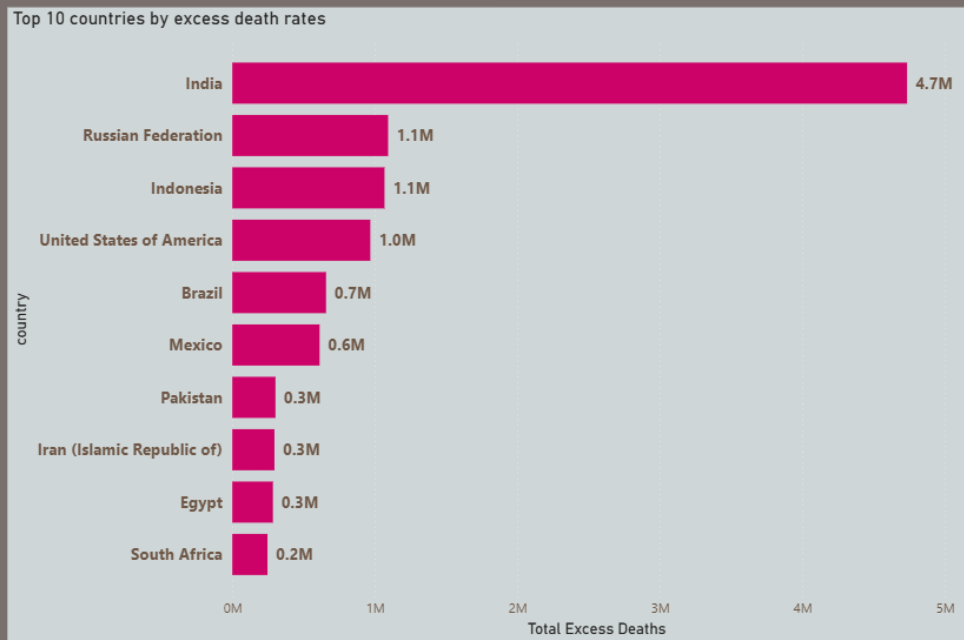
... Processed KPI files uploaded to S3!

- Files added in Processed file automatically

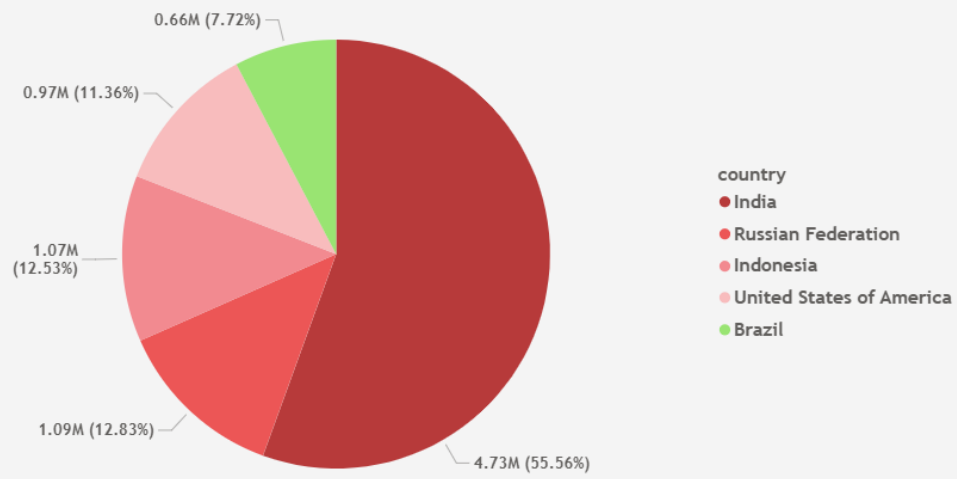
The screenshot shows the Amazon S3 console interface. The left sidebar contains navigation options like Buckets, Access management and security, and Storage management and insights. The main area displays the 'processed/' bucket. At the top, there are buttons for 'Copy S3 URI', 'Copy URL', 'Download', 'Open in', 'Delete', 'Actions', 'Create folder', and 'Upload'. Below these, a table lists the objects in the bucket:

| Name                | Type | Last modified                          | Size    | Storage class |
|---------------------|------|----------------------------------------|---------|---------------|
| country_age_kpi.csv | csv  | December 7, 2025, 18:02:08 (UTC+05:30) | 35.7 KB | Standard      |
| country_kpi.csv     | csv  | December 7, 2025, 18:02:07 (UTC+05:30) | 4.9 KB  | Standard      |
| country_sex_kpi.csv | csv  | December 7, 2025, 18:02:08 (UTC+05:30) | 9.4 KB  | Standard      |

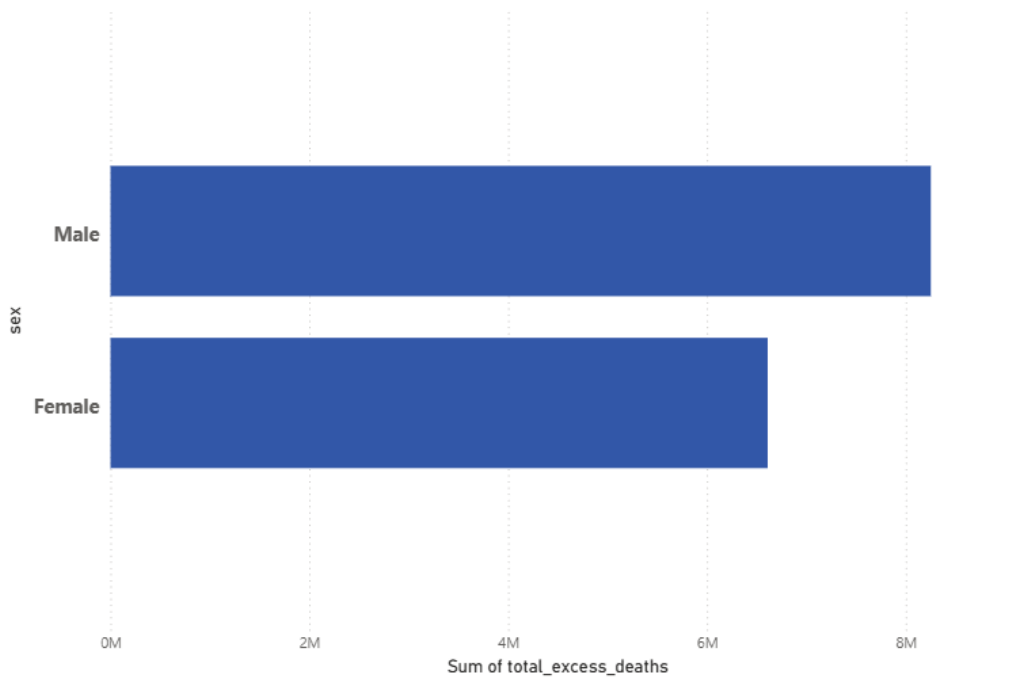
- Power Bi Visualisation



Share of Global Excess Deaths - Top 5 Countries



Sum of total\_excess\_deaths by sex



# RESULTS

The primary objective of this project was to process the WHO COVID-19 dataset using PySpark and store the transformed data in AWS S3 for future analytical use.

The results confirm that the workflow was successfully executed end-to-end.

After transformation, the dataset was uploaded as a Parquet file into the AWS S3 bucket. This ensures:

## → Secure and Scalable Cloud Storage

The data is stored safely in the cloud with high availability and can scale as the dataset grows.

## → Improved Performance for Analytics

Parquet format reduces storage size and enhances query performance, which is ideal for Big Data environments.

## → Reusable Data for Future Applications

The stored dataset can now be accessed by BI tools like Power BI, data engineers, machine learning models, or any AWS analytics service without re-processing.

## → Supports Efficient Data Sharing

Teams can easily collaborate and reuse the processed dataset without accessing the raw file again.

Overall, the project successfully demonstrates a real-world data engineering pipeline from dataset ingestion to cloud-based storage, making the data ready for analytics and visualization.

## REFERENCES

- **World Health Organization.** (n.d.). *Global excess deaths associated with COVID-19: Modelled estimates*. Retrieved from <https://www.who.int/data/sets/global-excess-deaths-associated-with-covid-19-modelled-estimates>
- **Amazon Web Services.** (n.d.). *AWS Documentation*. Retrieved from <https://docs.aws.amazon.com/>